NECO Individual Project: Real vs AI-Generated Image Classification

Comparing MLP (with HOG features) vs CNN (end-to-end) using PyTorch

by Sirapob Charoenchaipiyakul

Dataset: GRAVEX-200K (Full samples 200k images)
Run on: HPC (High performance computing provided by City's, University of London)

Section 0 — Imports and Configuration

In [1]:
```
# Standard library
import os, copy, json, pickle, random, time, itertools, warnings
warnings.filterwarnings('ignore')

# Numerical / data
import numpy as np
import pandas as pd

# Image processing
from PIL import Image
from skimage.feature import hog

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms

# Scikit-learn
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, f1_score,
    precision_score, recall_score
)
from sklearn.preprocessing import StandardScaler

# Plotting
import matplotlib
matplotlib.use('Agg')          # Non-interactive backend — required for HPC (no display)
import matplotlib.pyplot as plt
import seaborn as sns

# Reproducibility
# Fix all random seeds upfront so results are reproducible across runs.
# This covers Python's random, NumPy, and PyTorch (CPU side).
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# ─────────────────────────────────────────────────────────────────────────────
# HPC PATH CONFIGURATION
# Dataset must be uploaded to this exact path on Hyperion before submitting.
# All paths are absolute HPC paths — the dataset was uploaded to Hyperion
# before running this job. The layout inside DATASET_ROOT is:
#   my_real_vs_ai_dataset/ai_images/  — AI-generated images
#   my_real_vs_ai_dataset/real/        — real photographs
#   train_labels.csv / val_labels.csv / test_labels.csv
# Expected layout inside DATASET_ROOT:
#     my_real_vs_ai_dataset/
#         ai_images/
#         real/
#     test_labels.csv
#     train_labels.csv
#     val_labels.csv
# ─────────────────────────────────────────────────────────────────────────────
DATASET_ROOT = "/users/adhp470/neco_project/Dataset"

# Image sub-folders
AI_IMG_DIR   = os.path.join(DATASET_ROOT, 'my_real_vs_ai_dataset', 'ai_images')
REAL_IMG_DIR = os.path.join(DATASET_ROOT, 'my_real_vs_ai_dataset', 'real')

# CSV split files
TRAIN_CSV = os.path.join(DATASET_ROOT, 'train_labels.csv')
VAL_CSV   = os.path.join(DATASET_ROOT, 'val_labels.csv')
TEST_CSV  = os.path.join(DATASET_ROOT, 'test_labels.csv')

# Output folder — all saved files will be prefixed with HPC_
OUT_DIR = "/users/adhp470/neco_project/outputs"
os.makedirs(OUT_DIR, exist_ok=True)

# HPC prefix applied to every saved file
HPC_PREFIX = "HPC_"

# 128x128 was chosen as the input resolution — large enough to preserve
# meaningful texture detail for HOG, but small enough to keep memory
# and compute costs manageable across 200k images.
IMG_SIZE = 128
print('All imports OK.')
```

```
print(f'Dataset root : {DATASET_ROOT}')
print(f'Output dir   : {OUT_DIR}')
print(f'File prefix  : {HPC_PREFIX}')
```

```
All imports OK.
Dataset root : /users/adhp470/neco_project/Dataset
Output dir   : /users/adhp470/neco_project/outputs
File prefix  : HPC_
```

Section 1 — Dataset Loading and Analysis

In [2]:
```
# Load and normalise CSV label files

# The CSVs may contain extra metadata columns depending on how they were exported.
# normalise_df strips everything down to just filename and label to keep
# things consistent regardless of the source format.
def normalise_df(df):
    """Keep only filename and label columns; label encoded as int."""
    df = df.copy()
    print('  Raw columns :', df.columns.tolist())
    print('  Label sample:', df['label'].iloc[:5].tolist())
    df = df[['filename', 'label']].reset_index(drop=True)
    df['label'] = df['label'].astype(int)
    print(f'  → kept {len(df):,} rows, labels: {sorted(df["label"].unique())}')
    return df

print('=== train ===')
train_df = normalise_df(pd.read_csv(TRAIN_CSV))
print('=== val ===')
val_df   = normalise_df(pd.read_csv(VAL_CSV))
print('=== test ===')
test_df  = normalise_df(pd.read_csv(TEST_CSV))
print(f'\nTrain: {len(train_df):,}  Val: {len(val_df):,}  Test: {len(test_df):,}')
```

```
=== train ===
  Raw columns : ['Unnamed: 0', 'filename', 'label', 'resolution', 'image_size']
  Label sample: [1, 1, 1, 1, 1]
  → kept 140,000 rows, labels: [np.int64(0), np.int64(1)]
=== val ===
  Raw columns : ['Unnamed: 0', 'filename', 'label', 'resolution', 'image_size']
  Label sample: [1, 1, 1, 1, 1]
  → kept 40,000 rows, labels: [np.int64(0), np.int64(1)]
=== test ===
  Raw columns : ['Unnamed: 0', 'filename', 'label', 'resolution', 'image_size']
  Label sample: [1, 1, 1, 1, 1]
  → kept 20,000 rows, labels: [np.int64(0), np.int64(1)]

Train: 140,000  Val: 40,000  Test: 20,000
```

In [3]:
```
# Path resolver — images live in ai_images/ or real/ sub-folders

def get_full_path(filename):
    """
    Given a bare filename from the CSV, search ai_images/ and real/ sub-folders.
    Returns the full absolute path to the image file.
    """
    for folder in (AI_IMG_DIR, REAL_IMG_DIR):
        candidate = os.path.join(folder, filename)
        if os.path.isfile(candidate):
            return candidate
    raise FileNotFoundError(
        f'{filename!r} not found in ai_images/ or real/\n'
        f'  Checked: {AI_IMG_DIR}\n'
        f'           {REAL_IMG_DIR}'
    )

# Images are stored in one of two sub-folders depending on their class.
# Rather than encoding which folder each image belongs to in the CSV,
# get_full_path() just checks both and returns whichever exists.
# This keeps the CSV format simple and avoids path errors.
# Quick sanity check
sample_file = train_df.iloc[0]['filename']
print(f'Sample filename : {sample_file}')
print(f'Resolved to     : {get_full_path(sample_file)}')
```

```
Sample filename : 03428.jpg
Resolved to     : /users/adhp470/neco_project/Dataset/my_real_vs_ai_dataset/real/03428.jpg
```

In [4]:
```
# Class distribution summary

# Check class balance across all three splits before training.
# A heavily imbalanced dataset would make accuracy a misleading metric,
# so this is an important sanity check before committing to the pipeline.
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, (df, name) in zip(axes, [
        (train_df, 'Train'), (val_df, 'Validation'), (test_df, 'Test')]):
    counts = df['label'].value_counts().sort_index()
    bars = ax.bar(['Real', 'AI'], counts.values,
                  color=['steelblue', 'tomato'], edgecolor='black', width=0.5)
    for bar, v in zip(bars, counts.values):
        ax.text(bar.get_x() + bar.get_width()/2,
                v + counts.max()*0.01,
                f'{v:,}', ha='center', va='bottom', fontsize=9)
    ax.set_title(f'{name} split')
    ax.set_ylabel('Images')
    ax.set_ylim(0, counts.max() * 1.12)

fig.suptitle('Figure 1 — Class Distribution', fontsize=13, fontweight='bold')
```

```
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig1_class_distribution.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig1_class_distribution.png
```

In [5]:
```
# Show a sample of real vs AI images

# Visually inspect a handful of images from each class to confirm
# the data loaded correctly and to get a feel for how different
# real and AI-generated images actually look.
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for row_idx, (lbl, name) in enumerate([(0, 'Real'), (1, 'AI')]):
    sample_rows = train_df[train_df['label'] == lbl].sample(5, random_state=SEED)
    for col_idx, (_, row) in enumerate(sample_rows.iterrows()):
        ax = axes[row_idx, col_idx]
        ax.imshow(Image.open(get_full_path(row['filename'])).convert('RGB')
                  .resize((IMG_SIZE, IMG_SIZE)))
        ax.set_title(name, fontsize=9,
                     color='steelblue' if lbl == 0 else 'tomato')
        ax.axis('off')

fig.suptitle('Sample Images: Real (top) vs AI (bottom)',
             fontsize=12, fontweight='bold')
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig_sample_images.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig_sample_images.png
```

Section 2 — Dataset Classes, HOG Extraction, and Subsampling

In [6]:
```
# 2.1  Raw-pixel Dataset class (used by CNN DataLoaders)

class RealVsAIDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df        = dataframe.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row   = self.df.iloc[idx]
        path  = get_full_path(row['filename'])
        label = int(row['label'])
        img   = Image.open(path).convert('RGB')
        if self.transform:
            img = self.transform(img)
        return img, label

# Normalise pixel values to [-1, 1] by subtracting 0.5 and dividing by 0.5.
# This centres the distribution around zero, which helps gradient-based
# optimisers converge faster and more stably.
NORM_MEAN = [0.5, 0.5, 0.5]
NORM_STD  = [0.5, 0.5, 0.5]

# The training transform includes data augmentation — random horizontal flips,
# small rotations, and slight colour jitter. These artificially expand the
# effective training set and help prevent overfitting on the CNN.
# The eval transform is deterministic (no augmentation) so results are consistent.
train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=NORM_MEAN, std=NORM_STD),
])

eval_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=NORM_MEAN, std=NORM_STD),
])

print('RealVsAIDataset and transforms defined.')
```

```
RealVsAIDataset and transforms defined.
```

In [7]:
```
# 2.2  HOG feature extraction

# HOG (Histogram of Oriented Gradients) is the hand-crafted feature descriptor
# used as input to the MLP. It captures local edge and gradient structure
# across small spatial cells, which encodes texture and shape information
# without the MLP needing to learn spatial features from scratch.
def extract_hog_features(img_pil, orientations=9,
                         pixels_per_cell=(8, 8), cells_per_block=(2, 2)):
    img = img_pil.resize((IMG_SIZE, IMG_SIZE)).convert('RGB')
    img_array = np.array(img, dtype=np.float32) / 255.0
    # HOG parameter choices:
    #   orientations=9  — divides 180° into 9 bins (20° each), standard setting
    #   pixels_per_cell=(8,8) — each cell covers an 8x8 pixel region
    #   cells_per_block=(2,2) — normalise over 2x2 cell neighbourhoods for contrast invariance
```

```
    #   channel_axis=-1 — tells skimage the colour channel is the last dimension
    features = hog(
        img_array,
        orientations=orientations,
        pixels_per_cell=pixels_per_cell,
        cells_per_block=cells_per_block,
        channel_axis=-1,
        feature_vector=True,
    )
    return features.astype(np.float32)

# Compute the HOG vector length once using a blank dummy image
# so the MLP input dimension can be set automatically.
_dummy = Image.new('RGB', (IMG_SIZE, IMG_SIZE))
HOG_FEATURE_DIM = len(extract_hog_features(_dummy))
print(f'HOG feature vector length: {HOG_FEATURE_DIM}')
```

```
HOG feature vector length: 8100
```

In [8]:

```
# 2.3  HOG extraction — full splits

def build_hog_arrays(dataframe, desc=''):
    """Extract HOG for every row; return (X float32, y int64)."""
    X, y = [], []
    n         = len(dataframe)
    filenames = dataframe['filename'].tolist()
    labels    = dataframe['label'].tolist()
    for i, (fname, lbl) in enumerate(zip(filenames, labels)):
        if i % 2000 == 0:
            print(f'  {desc}: {i:,}/{n:,}', end='\r')
        img = Image.open(get_full_path(fname)).convert('RGB')
        X.append(extract_hog_features(img))
        y.append(int(lbl))
    print(f'  {desc}: {n:,}/{n:,} — done.     ')
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.int64)

# Search subsets (small — only for hyperparameter search phases)
# Hyperparameter search runs on a 5,000/1,000 stratified subset rather than
# the full dataset. This lets me evaluate 20+ configurations quickly
# without waiting hours per trial. The final model is then trained on
# the full data using the best config found here.
SEARCH_TRAIN, SEARCH_VAL = 5000, 1000

search_train = pd.concat([
    train_df[train_df['label'] == 0].sample(SEARCH_TRAIN // 2, random_state=SEED),
    train_df[train_df['label'] == 1].sample(SEARCH_TRAIN // 2, random_state=SEED),
]).sample(frac=1, random_state=SEED).reset_index(drop=True)

search_val = pd.concat([
    val_df[val_df['label'] == 0].sample(SEARCH_VAL // 2, random_state=SEED),
    val_df[val_df['label'] == 1].sample(SEARCH_VAL // 2, random_state=SEED),
]).sample(frac=1, random_state=SEED).reset_index(drop=True)

print(f'Search subset: {len(search_train):,} train / {len(search_val):,} val')

# Full splits for final training and evaluation
# For the final training run, use all available data — no subsampling.
train_sub = train_df.copy()
val_sub   = val_df.copy()
test_sub  = test_df.copy()
print(f'Final splits: {len(train_sub):,} train / {len(val_sub):,} val / {len(test_sub):,} test')

# Extract HOG for full splits
print('\nExtracting HOG features for full splits (this will take several minutes ...')
X_train_hog, y_train = build_hog_arrays(train_sub, 'Train')
X_val_hog,   y_val   = build_hog_arrays(val_sub,   'Val')
X_test_hog,  y_test  = build_hog_arrays(test_sub,  'Test')

# Fit StandardScaler on training HOG features only, then apply the same
# transformation to val and test. Fitting on val/test would be data leakage.
# This ensures all splits are normalised using training statistics.
scaler      = StandardScaler()
X_train_hog = scaler.fit_transform(X_train_hog)
X_val_hog   = scaler.transform(X_val_hog)
X_test_hog  = scaler.transform(X_test_hog)
print(f'HOG arrays ready. Train shape: {X_train_hog.shape}')

# Extract HOG for search subsets
print('\nExtracting HOG for search subsets...')
X_search_train_hog, y_search_train = build_hog_arrays(search_train, 'Search train')
X_search_val_hog,   y_search_val   = build_hog_arrays(search_val,   'Search val')
# Apply the same training scaler to the search subsets — not fit_transform,
# since the scaler was already fitted on the full training set above.
X_search_train_hog = scaler.transform(X_search_train_hog)
X_search_val_hog   = scaler.transform(X_search_val_hog)
print(f'Search HOG ready. Shape: {X_search_train_hog.shape}')
```

```
Search subset: 5,000 train / 1,000 val
Final splits: 140,000 train / 40,000 val / 20,000 test

Extracting HOG features for full splits (this will take several minutes ...
  Train: 0/140,000
  Train: 2,000/140,000
  Train: 4,000/140,000
  Train: 6,000/140,000
  Train: 8,000/140,000
  Train: 10,000/140,000
  Train: 12,000/140,000
  Train: 14,000/140,000
  Train: 16,000/140,000
  Train: 18,000/140,000
```

```
  Train: 20,000/140,000
  Train: 22,000/140,000
  Train: 24,000/140,000
  Train: 26,000/140,000
  Train: 28,000/140,000
  Train: 30,000/140,000
  Train: 32,000/140,000
  Train: 34,000/140,000
  Train: 36,000/140,000
  Train: 38,000/140,000
  Train: 40,000/140,000
  Train: 42,000/140,000
  Train: 44,000/140,000
  Train: 46,000/140,000
  Train: 48,000/140,000
  Train: 50,000/140,000
  Train: 52,000/140,000
  Train: 54,000/140,000
  Train: 56,000/140,000
  Train: 58,000/140,000
  Train: 60,000/140,000
  Train: 62,000/140,000
  Train: 64,000/140,000
  Train: 66,000/140,000
  Train: 68,000/140,000
  Train: 70,000/140,000
  Train: 72,000/140,000
  Train: 74,000/140,000
  Train: 76,000/140,000
  Train: 78,000/140,000
  Train: 80,000/140,000
  Train: 82,000/140,000
  Train: 84,000/140,000
  Train: 86,000/140,000
  Train: 88,000/140,000
  Train: 90,000/140,000
  Train: 92,000/140,000
  Train: 94,000/140,000
  Train: 96,000/140,000
  Train: 98,000/140,000
  Train: 100,000/140,000
  Train: 102,000/140,000
  Train: 104,000/140,000
  Train: 106,000/140,000
  Train: 108,000/140,000
  Train: 110,000/140,000
  Train: 112,000/140,000
  Train: 114,000/140,000
  Train: 116,000/140,000
  Train: 118,000/140,000
  Train: 120,000/140,000
  Train: 122,000/140,000
  Train: 124,000/140,000
  Train: 126,000/140,000
  Train: 128,000/140,000
  Train: 130,000/140,000
  Train: 132,000/140,000
  Train: 134,000/140,000
  Train: 136,000/140,000
  Train: 138,000/140,000
  Train: 140,000/140,000 — done.
  Val: 0/40,000
  Val: 2,000/40,000
  Val: 4,000/40,000
  Val: 6,000/40,000
  Val: 8,000/40,000
  Val: 10,000/40,000
  Val: 12,000/40,000
  Val: 14,000/40,000
  Val: 16,000/40,000
  Val: 18,000/40,000
  Val: 20,000/40,000
  Val: 22,000/40,000
  Val: 24,000/40,000
  Val: 26,000/40,000
  Val: 28,000/40,000
  Val: 30,000/40,000
  Val: 32,000/40,000
  Val: 34,000/40,000
  Val: 36,000/40,000
  Val: 38,000/40,000
  Val: 40,000/40,000 — done.
  Test: 0/20,000
  Test: 2,000/20,000
  Test: 4,000/20,000
  Test: 6,000/20,000
  Test: 8,000/20,000
  Test: 10,000/20,000
  Test: 12,000/20,000
  Test: 14,000/20,000
  Test: 16,000/20,000
  Test: 18,000/20,000
  Test: 20,000/20,000 — done.
HOG arrays ready. Train shape: (140000, 8100)

Extracting HOG for search subsets...
  Search train: 0/5,000
  Search train: 2,000/5,000
  Search train: 4,000/5,000
```

```
  Search train: 5,000/5,000 — done.
  Search val: 0/1,000
  Search val: 1,000/1,000 — done.
Search HOG ready. Shape: (5000, 8100)
```

In [9]:
```python
# 2.4  Visualise HOG on a real vs AI image

# Re-import hog with visualize support (aliased to avoid conflict with the
# extract_hog_features function defined above which uses the standard import).
from skimage.feature import hog as sk_hog

fig, axes = plt.subplots(2, 2, figsize=(9, 9))
for row_idx, (lbl, name) in enumerate([(0, 'Real'), (1, 'AI')]):
    row = train_sub[train_sub['label'] == lbl].iloc[0]
    img_array = np.array(
        Image.open(get_full_path(row['filename']))
             .convert('RGB').resize((IMG_SIZE, IMG_SIZE)),
        dtype=np.float32
    ) / 255.0

    # visualise=True returns a rendered gradient image alongside the feature vector.
    # The underscore discards the feature vector — only the visual is needed here.
    _, hog_image = sk_hog(
        img_array, orientations=9,
        pixels_per_cell=(8, 8), cells_per_block=(2, 2),
        channel_axis=-1, visualize=True
    )

    axes[row_idx, 0].imshow(img_array)
    axes[row_idx, 0].set_title(f'{name} — original')
    axes[row_idx, 0].axis('off')

    axes[row_idx, 1].imshow(hog_image, cmap='inferno')
    axes[row_idx, 1].set_title(f'{name} — HOG')
    axes[row_idx, 1].axis('off')

fig.suptitle('HOG Feature Visualisation: Real vs AI', fontsize=12, fontweight='bold')
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig_hog_visualisation.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig_hog_visualisation.png
```

Section 3 — Model Architectures

In [10]:
```python
# 3.1  MLP architecture

# The MLP takes a HOG feature vector as input and classifies it as real or AI.
# The number of hidden layers and their sizes are treated as hyperparameters
# and tuned via search in Section 5 — the architecture is built dynamically
# from the hidden_dims list rather than being hardcoded.
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, dropout=0.3):
        super().__init__()
        layers   = []
        prev_dim = input_dim
        for h_dim in hidden_dims:
            layers += [
                nn.Linear(prev_dim, h_dim),
                # BatchNorm normalises activations across the batch before passing them
                # through ReLU. This stabilises training, reduces sensitivity to the
                # learning rate, and generally speeds up convergence.
                nn.BatchNorm1d(h_dim),
                nn.ReLU(),
                # Dropout randomly zeros a fraction of neurons during training,
                # forcing the network not to rely on any single feature and
                # acting as a regulariser to reduce overfitting.
                nn.Dropout(p=dropout),
            ]
            prev_dim = h_dim
        # Final layer outputs 2 logits (one per class: real vs AI).
        # CrossEntropyLoss applies softmax internally, so no activation here.
        layers.append(nn.Linear(prev_dim, 2))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)

_m = MLP(HOG_FEATURE_DIM, [512, 256])
print('MLP output shape:', _m(torch.randn(4, HOG_FEATURE_DIM)).shape)
print(f'MLP parameters:  {sum(p.numel() for p in _m.parameters()):,}')
```

```
MLP output shape: torch.Size([4, 2])
MLP parameters:  4,281,090
```

In [11]:
```python
# 3.2  CNN architecture

# The CNN operates directly on raw pixel data — no hand-crafted features.
# Three convolutional blocks progressively extract higher-level features,
# each doubling the channel count while halving spatial dimensions via MaxPool.
# A small fully-connected head then maps the learned representations to 2 classes.
class CNN(nn.Module):
    def __init__(self, num_filters=32, dropout=0.3, img_size=IMG_SIZE):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1: input is 3-channel RGB at 128x128 → output is num_filters channels at 64x64.
```

```
            nn.Conv2d(3, num_filters, kernel_size=3, padding=1),
            nn.BatchNorm2d(num_filters), nn.ReLU(inplace=True), nn.MaxPool2d(2),
            # Block 2: num_filters channels at 64x64 → num_filters*2 channels at 32x32.
            nn.Conv2d(num_filters, num_filters * 2, kernel_size=3, padding=1),
            nn.BatchNorm2d(num_filters * 2), nn.ReLU(inplace=True), nn.MaxPool2d(2),
            # Block 3: num_filters*2 channels at 32x32 → num_filters*4 channels at 16x16.
            nn.Conv2d(num_filters * 2, num_filters * 4, kernel_size=3, padding=1),
            nn.BatchNorm2d(num_filters * 4), nn.ReLU(inplace=True), nn.MaxPool2d(2),
        )
        # After three MaxPool2d(2) layers, each halving the spatial dims,
        # the feature map is (img_size // 8) x (img_size // 8) = 16x16 for IMG_SIZE=128.
        # flat_size is the total number of values passed to the FC head.
        flat_size = num_filters * 4 * (img_size // 8) ** 2
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(flat_size, 256), nn.ReLU(inplace=True),
            nn.Dropout(p=dropout),
            nn.Linear(256, 2),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

# Quick output shape check — confirms the architecture is correctly wired
# before starting any training.
_c = CNN(num_filters=32, dropout=0.3, img_size=IMG_SIZE)
print('CNN output shape:', _c(torch.randn(4, 3, IMG_SIZE, IMG_SIZE)).shape)
print(f'CNN parameters:   {sum(p.numel() for p in _c.parameters()):,}')
```

```
CNN output shape: torch.Size([4, 2])
CNN parameters:   8,483,074
```

Section 4 — Training and Evaluation Utilities

In [12]:

```
# 4.1  Generic one-epoch train and eval functions

# train_epoch and eval_epoch are kept as separate functions so they can be
# reused cleanly across the hyperparameter search (Section 5) and the
# full training run (Section 6) without duplicating logic.
def train_epoch(model, loader, criterion, optimiser):
    # model.train() enables Dropout and switches BatchNorm to use batch statistics.
    # Must be called at the start of every training epoch.
    model.train()
    total_loss, correct, n = 0.0, 0, 0
    for X_batch, y_batch in loader:
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
        # PyTorch accumulates gradients by default, so they must be explicitly
        # cleared before each backward pass to avoid corrupting the weight updates.
        optimiser.zero_grad()
        logits = model(X_batch)
        loss   = criterion(logits, y_batch)
        # Backpropagation: compute gradients of the loss w.r.t. all parameters.
        loss.backward()
        # Apply the Adam update rule to all parameters using the computed gradients.
        optimiser.step()
        total_loss += loss.item() * len(y_batch)
        correct    += (logits.argmax(1) == y_batch).sum().item()
        n          += len(y_batch)
    return total_loss / n, correct / n


def eval_epoch(model, loader, criterion):
    # model.eval() disables Dropout and switches BatchNorm to use its running
    # statistics (accumulated during training) rather than batch statistics.
    # This gives deterministic, stable outputs during evaluation.
    model.eval()
    total_loss, correct, n = 0.0, 0, 0
    all_preds, all_labels  = [], []
    # no_grad() disables gradient tracking during evaluation — not needed here
    # and disabling it saves memory and speeds up inference.
    with torch.no_grad():
        for X_batch, y_batch in loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            logits  = model(X_batch)
            loss    = criterion(logits, y_batch)
            total_loss += loss.item() * len(y_batch)
            preds       = logits.argmax(1)
            correct    += (preds == y_batch).sum().item()
            n          += len(y_batch)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(y_batch.cpu().numpy())
    return total_loss/n, correct/n, np.array(all_preds), np.array(all_labels)


# 4.2  DataLoader factories

# num_workers=4 tells PyTorch to use 4 parallel CPU processes for data loading,
# so the GPU is never waiting idle for the next batch.
# pin_memory=True in make_cnn_loaders speeds up CPU-to-GPU data transfer on HPC.
def make_hog_loaders(batch_size=128):
    def _dl(X, y, shuffle):
        ds = torch.utils.data.TensorDataset(
            torch.tensor(X, dtype=torch.float32),
            torch.tensor(y, dtype=torch.long)
        )
        return DataLoader(ds, batch_size=batch_size, shuffle=shuffle, num_workers=4)
    return (_dl(X_train_hog, y_train, True),
```

```
            _dl(X_val_hog,   y_val,   False),
            _dl(X_test_hog,  y_test,  False))


def make_cnn_loaders(batch_size=64):
    train_ds = RealVsAIDataset(train_sub, transform=train_transform)
    val_ds   = RealVsAIDataset(val_sub,   transform=eval_transform)
    test_ds  = RealVsAIDataset(test_sub,  transform=eval_transform)
    kw = dict(num_workers=4, pin_memory=True)   # HPC: multi-worker loading + pin memory for GPU
    return (DataLoader(train_ds, batch_size=batch_size, shuffle=True,  **kw),
            DataLoader(val_ds,   batch_size=batch_size, shuffle=False, **kw),
            DataLoader(test_ds,  batch_size=batch_size, shuffle=False, **kw))


def make_search_hog_loaders(batch_size=128):
    def _dl(X, y, shuffle):
        ds = torch.utils.data.TensorDataset(
            torch.tensor(X, dtype=torch.float32),
            torch.tensor(y, dtype=torch.long)
        )
        return DataLoader(ds, batch_size=batch_size, shuffle=shuffle, num_workers=4)
    return (_dl(X_search_train_hog, y_search_train, True),
            _dl(X_search_val_hog,   y_search_val,   False))


print('Training utilities defined.')
```

```
Training utilities defined.
```

Section 5 — Hyperparameter Search

In [13]:
```
# 5.1  Quick-train helper for search trials

# quick_train runs a short trial (5 epochs by default) and returns the best
# validation accuracy seen. It's used inside the search loops to evaluate
# each configuration cheaply before committing to a full training run.
# Early stopping with patience=3 cuts wasted compute on bad configs.
def quick_train(model, train_loader, val_loader,
                lr, weight_decay, n_epochs=5, patience=3):
    model = model.to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimiser = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    best_val_acc, no_improve = 0.0, 0
    for _ in range(n_epochs):
        train_epoch(model, train_loader, criterion, optimiser)
        _, val_acc, _, _ = eval_epoch(model, val_loader, criterion)
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            no_improve   = 0
        else:
            no_improve += 1
            if no_improve >= patience:
                break
    return best_val_acc
```

In [14]:
```
# HPC: use CUDA (A100). Falls back to CPU if GPU not allocated.
# Set DEVICE here rather than at the top of the notebook because DEVICE
# is only needed once training starts — defining it earlier would have
# no effect and might be confusing.
if torch.cuda.is_available():
    DEVICE = torch.device("cuda")
    torch.cuda.manual_seed(SEED)
    print(f"Using device: {DEVICE} — {torch.cuda.get_device_name(0)}")
else:
    DEVICE = torch.device("cpu")
    print(f"Using device: {DEVICE} (no GPU detected)")
```

```
Using device: cuda — NVIDIA A100-PCIE-40GB
```

In [15]:
```
# 5.2  Random Search for MLP

# Random search over 20 configurations gives a good spread across the
# search space without the exponential cost of a full grid search.
# The search space covers: hidden layer configurations, learning rate,
# weight decay, dropout rate, and batch size.
N_RANDOM_TRIALS = 20

mlp_random_space = {
    'hidden_dims': [
        [256], [512],
        [512, 256], [1024, 512],
        [512, 256, 128], [1024, 512, 256],
    ],
    'lr':           [1e-4, 5e-4, 1e-3, 5e-3],
    'weight_decay': [0, 1e-5, 1e-4, 1e-3],
    'dropout':      [0.1, 0.2, 0.3, 0.5],
    'batch_size':   [64, 128, 256],
}

mlp_random_results = []
print(f'Running {N_RANDOM_TRIALS} random MLP trials (on {SEARCH_TRAIN} images)...\n')

for trial in range(N_RANDOM_TRIALS):
    # Sample one value per hyperparameter uniformly at random.
    cfg = {k: random.choice(v) for k, v in mlp_random_space.items()}
    tr_l, val_l = make_search_hog_loaders(batch_size=cfg['batch_size'])
    # Re-seed before each trial so weight initialisation is the same across
    # trials — isolating the effect of hyperparameters from random init variance.
```

```
    torch.manual_seed(SEED)
    model   = MLP(HOG_FEATURE_DIM, cfg['hidden_dims'], cfg['dropout'])
    t0      = time.time()
    val_acc = quick_train(model, tr_l, val_l, lr=cfg['lr'], weight_decay=cfg['weight_decay'])
    cfg['val_acc'] = val_acc
    mlp_random_results.append(cfg)
    print(f'  Trial {trial+1:2d}: val_acc={val_acc:.4f} | '
          f'hidden={cfg["hidden_dims"]} lr={cfg["lr"]} '
          f'wd={cfg["weight_decay"]} drop={cfg["dropout"]} '
          f'bs={cfg["batch_size"]} ({time.time()-t0:.1f}s)')

mlp_random_results.sort(key=lambda x: x['val_acc'], reverse=True)
print(f'\nTop 3 MLP random configs:')
for r in mlp_random_results[:3]:
    print(f'  val_acc={r["val_acc"]:.4f} | {r}')
```

```
Running 20 random MLP trials (on 5000 images)...

  Trial  1: val_acc=0.7620 | hidden=[1024, 512, 256] lr=0.0001 wd=0 drop=0.3 bs=64 (9.5s)
  Trial  2: val_acc=0.7600 | hidden=[512] lr=0.0005 wd=0 drop=0.1 bs=256 (8.1s)
  Trial  3: val_acc=0.7580 | hidden=[1024, 512] lr=0.0001 wd=0 drop=0.1 bs=64 (8.9s)
  Trial  4: val_acc=0.7480 | hidden=[512] lr=0.0001 wd=1e-05 drop=0.5 bs=64 (8.7s)
  Trial  5: val_acc=0.7580 | hidden=[1024, 512] lr=0.001 wd=0 drop=0.2 bs=256 (8.2s)
  Trial  6: val_acc=0.7560 | hidden=[1024, 512] lr=0.001 wd=0.0001 drop=0.2 bs=64 (9.0s)
  Trial  7: val_acc=0.7400 | hidden=[512, 256] lr=0.0001 wd=0 drop=0.5 bs=64 (8.9s)
  Trial  8: val_acc=0.7590 | hidden=[512, 256] lr=0.001 wd=0.0001 drop=0.1 bs=256 (8.0s)
  Trial  9: val_acc=0.7440 | hidden=[1024, 512] lr=0.0001 wd=0.001 drop=0.1 bs=256 (8.1s)
  Trial 10: val_acc=0.7480 | hidden=[512, 256] lr=0.001 wd=1e-05 drop=0.1 bs=64 (8.8s)
  Trial 11: val_acc=0.7570 | hidden=[1024, 512, 256] lr=0.0005 wd=0.0001 drop=0.1 bs=64 (9.1s)
  Trial 12: val_acc=0.7450 | hidden=[256] lr=0.005 wd=0.0001 drop=0.5 bs=256 (8.1s)
  Trial 13: val_acc=0.7530 | hidden=[512, 256] lr=0.0005 wd=0.0001 drop=0.3 bs=64 (8.8s)
  Trial 14: val_acc=0.7620 | hidden=[1024, 512, 256] lr=0.001 wd=0 drop=0.2 bs=256 (8.1s)
  Trial 15: val_acc=0.7670 | hidden=[1024, 512, 256] lr=0.0005 wd=1e-05 drop=0.5 bs=128 (8.6s)
  Trial 16: val_acc=0.7530 | hidden=[512, 256] lr=0.0005 wd=0.0001 drop=0.1 bs=64 (8.9s)
  Trial 17: val_acc=0.7640 | hidden=[256] lr=0.001 wd=0.001 drop=0.3 bs=64 (8.7s)
  Trial 18: val_acc=0.7560 | hidden=[512] lr=0.001 wd=1e-05 drop=0.5 bs=128 (8.3s)
  Trial 19: val_acc=0.7450 | hidden=[1024, 512, 256] lr=0.005 wd=1e-05 drop=0.3 bs=64 (9.2s)
  Trial 20: val_acc=0.7510 | hidden=[512] lr=0.001 wd=0.001 drop=0.5 bs=128 (8.4s)

Top 3 MLP random configs:
  val_acc=0.7670 | {'hidden_dims': [1024, 512, 256], 'lr': 0.0005, 'weight_decay': 1e-05, 'dropout': 0.5, 'batch_size': 128, 'val_ac
c': 0.767}
  val_acc=0.7640 | {'hidden_dims': [256], 'lr': 0.001, 'weight_decay': 0.001, 'dropout': 0.3, 'batch_size': 64, 'val_acc': 0.764}
  val_acc=0.7620 | {'hidden_dims': [1024, 512, 256], 'lr': 0.0001, 'weight_decay': 0, 'dropout': 0.3, 'batch_size': 64, 'val_acc': 0.
762}
```

In [16]:

```
# 5.3  Grid Search for MLP

best_r = mlp_random_results[0]
print(f'Best random config: {best_r}\n')

# Grid search refines around the best random config using a coarse-to-fine
# strategy: fix architecture and dropout at the best values found, then
# explore a 3-point range around the best learning rate and weight decay.
# This gives 9 combinations (3 lr x 3 wd) at lower cost than a full grid.
mlp_grid = {
    'hidden_dims': [best_r['hidden_dims']],
    'dropout':     [best_r['dropout']],
    'batch_size':  [best_r['batch_size']],
    'lr':          sorted({best_r['lr'] * 0.5, best_r['lr'], best_r['lr'] * 2.0}),
    'weight_decay': sorted({max(0, best_r['weight_decay'] * 0.1),
                            best_r['weight_decay'],
                            best_r['weight_decay'] * 10}),
}

mlp_grid_results = []
keys   = list(mlp_grid.keys())
combos = list(itertools.product(*mlp_grid.values()))
print(f'Running {len(combos)} MLP grid search trials (on {SEARCH_TRAIN} images)...\n')

for trial, values in enumerate(combos):
    cfg = dict(zip(keys, values))
    tr_l, val_l = make_search_hog_loaders(batch_size=cfg['batch_size'])
    torch.manual_seed(SEED)
    model   = MLP(HOG_FEATURE_DIM, cfg['hidden_dims'], cfg['dropout'])
    val_acc = quick_train(model, tr_l, val_l,
                          lr=cfg['lr'], weight_decay=cfg['weight_decay'], n_epochs=8)
    cfg['val_acc'] = val_acc
    mlp_grid_results.append(cfg)
    print(f'  Trial {trial+1:2d}: val_acc={val_acc:.4f} | '
          f'lr={cfg["lr"]:.2e} wd={cfg["weight_decay"]:.2e}')

mlp_grid_results.sort(key=lambda x: x['val_acc'], reverse=True)
BEST_MLP_CFG = mlp_grid_results[0]
print(f'\n* Best MLP config: {BEST_MLP_CFG}')
```

```
Best random config: {'hidden_dims': [1024, 512, 256], 'lr': 0.0005, 'weight_decay': 1e-05, 'dropout': 0.5, 'batch_size': 128, 'val_ac
c': 0.767}

Running 9 MLP grid search trials (on 5000 images)...

  Trial  1: val_acc=0.7540 | lr=2.50e-04 wd=1.00e-06
  Trial  2: val_acc=0.7580 | lr=2.50e-04 wd=1.00e-05
  Trial  3: val_acc=0.7570 | lr=2.50e-04 wd=1.00e-04
  Trial  4: val_acc=0.7560 | lr=5.00e-04 wd=1.00e-06
  Trial  5: val_acc=0.7670 | lr=5.00e-04 wd=1.00e-05
  Trial  6: val_acc=0.7660 | lr=5.00e-04 wd=1.00e-04
  Trial  7: val_acc=0.7580 | lr=1.00e-03 wd=1.00e-06
  Trial  8: val_acc=0.7610 | lr=1.00e-03 wd=1.00e-05
```

```
  Trial  9: val_acc=0.7490 | lr=1.00e-03 wd=1.00e-04

★ Best MLP config: {'hidden_dims': [1024, 512, 256], 'dropout': 0.5, 'batch_size': 128, 'lr': 0.0005, 'weight_decay': 1e-05, 'val_ac
c': 0.767}
```

In [17]:
```
# 5.4  Random Search for CNN

# The CNN search space is smaller than the MLP's because each CNN trial
# is more expensive — it involves loading raw images rather than pre-extracted
# HOG vectors. The num_filters parameter controls model capacity.
cnn_random_space = {
    'num_filters':  [16, 32, 64],
    'lr':           [1e-4, 5e-4, 1e-3],
    'weight_decay': [0, 1e-5, 1e-4],
    'dropout':      [0.2, 0.3, 0.5],
    'batch_size':   [32, 64],
}

search_train_ds = RealVsAIDataset(search_train, transform=train_transform)
search_val_ds   = RealVsAIDataset(search_val,   transform=eval_transform)

cnn_random_results = []
print(f'Running {N_RANDOM_TRIALS} random CNN trials (on {SEARCH_TRAIN} images, 5 epochs each)...\n')

for trial in range(N_RANDOM_TRIALS):
    cfg  = {k: random.choice(v) for k, v in cnn_random_space.items()}
    tr_l = DataLoader(search_train_ds, batch_size=cfg['batch_size'], shuffle=True,  num_workers=4)
    val_l = DataLoader(search_val_ds,  batch_size=cfg['batch_size'], shuffle=False, num_workers=4)
    torch.manual_seed(SEED)
    model   = CNN(cfg['num_filters'], cfg['dropout'], IMG_SIZE)
    t0      = time.time()
    val_acc = quick_train(model, tr_l, val_l,
                          lr=cfg['lr'], weight_decay=cfg['weight_decay'],
                          n_epochs=5, patience=2) # I can't do more than 3 epochs here due to computational constraint.
    cfg['val_acc'] = val_acc
    cnn_random_results.append(cfg)
    print(f'  Trial {trial+1:2d}: val_acc={val_acc:.4f} | '
          f'filters={cfg["num_filters"]} lr={cfg["lr"]} '
          f'wd={cfg["weight_decay"]} drop={cfg["dropout"]} '
          f'bs={cfg["batch_size"]} ({time.time()-t0:.1f}s)')

cnn_random_results.sort(key=lambda x: x['val_acc'], reverse=True)
print(f'\nTop 3 CNN random configs:')
for r in cnn_random_results[:3]:
    print(f'  val_acc={r["val_acc"]:.4f} | {r}')
```

```
Running 20 random CNN trials (on 5000 images, 5 epochs each)...

  Trial  1: val_acc=0.6640 | filters=16 lr=0.0001 wd=0.0001 drop=0.3 bs=32 (24.0s)
  Trial  2: val_acc=0.6400 | filters=16 lr=0.0001 wd=0 drop=0.5 bs=32 (23.7s)
  Trial  3: val_acc=0.6210 | filters=64 lr=0.0005 wd=0.0001 drop=0.2 bs=64 (24.7s)
  Trial  4: val_acc=0.6050 | filters=32 lr=0.001 wd=1e-05 drop=0.5 bs=64 (24.4s)
  Trial  5: val_acc=0.6430 | filters=64 lr=0.0001 wd=0.0001 drop=0.5 bs=32 (23.9s)
  Trial  6: val_acc=0.5930 | filters=64 lr=0.001 wd=1e-05 drop=0.5 bs=64 (24.2s)
  Trial  7: val_acc=0.6380 | filters=16 lr=0.0005 wd=1e-05 drop=0.2 bs=64 (24.0s)
  Trial  8: val_acc=0.6240 | filters=16 lr=0.001 wd=0.0001 drop=0.3 bs=32 (18.9s)
  Trial  9: val_acc=0.6260 | filters=64 lr=0.0001 wd=0.0001 drop=0.3 bs=32 (14.1s)
  Trial 10: val_acc=0.6220 | filters=16 lr=0.0005 wd=0 drop=0.5 bs=32 (23.6s)
  Trial 11: val_acc=0.6110 | filters=64 lr=0.0005 wd=1e-05 drop=0.2 bs=32 (23.7s)
  Trial 12: val_acc=0.6170 | filters=32 lr=0.0005 wd=0 drop=0.2 bs=32 (14.2s)
  Trial 13: val_acc=0.6400 | filters=64 lr=0.0001 wd=0 drop=0.5 bs=64 (24.1s)
  Trial 14: val_acc=0.6340 | filters=16 lr=0.001 wd=0 drop=0.2 bs=64 (19.0s)
  Trial 15: val_acc=0.6330 | filters=64 lr=0.0001 wd=1e-05 drop=0.5 bs=64 (23.9s)
  Trial 16: val_acc=0.6140 | filters=16 lr=0.001 wd=0.0001 drop=0.5 bs=32 (18.9s)
  Trial 17: val_acc=0.6240 | filters=64 lr=0.0005 wd=1e-05 drop=0.5 bs=64 (24.2s)
  Trial 18: val_acc=0.6130 | filters=32 lr=0.001 wd=1e-05 drop=0.2 bs=32 (24.0s)
  Trial 19: val_acc=0.6690 | filters=16 lr=0.0001 wd=1e-05 drop=0.2 bs=32 (23.6s)
  Trial 20: val_acc=0.6580 | filters=64 lr=0.0001 wd=0 drop=0.2 bs=32 (23.9s)

Top 3 CNN random configs:
  val_acc=0.6690 | {'num_filters': 16, 'lr': 0.0001, 'weight_decay': 1e-05, 'dropout': 0.2, 'batch_size': 32, 'val_acc': 0.669}
  val_acc=0.6640 | {'num_filters': 16, 'lr': 0.0001, 'weight_decay': 0.0001, 'dropout': 0.3, 'batch_size': 32, 'val_acc': 0.664}
  val_acc=0.6580 | {'num_filters': 64, 'lr': 0.0001, 'weight_decay': 0, 'dropout': 0.2, 'batch_size': 32, 'val_acc': 0.658}
```

In [18]:
```
# 5.5  Grid Search for CNN

best_rc = cnn_random_results[0]
print(f'Best CNN random config: {best_rc}\n')

# Same coarse-to-fine grid search strategy as for the MLP:
# fix the best architecture and dropout, then sweep lr and weight decay.
cnn_grid = {
    'num_filters': [best_rc['num_filters']],
    'dropout':     [best_rc['dropout']],
    'batch_size':  [best_rc['batch_size']],
    'lr':          sorted({best_rc['lr'] * 0.5, best_rc['lr'], best_rc['lr'] * 2.0}),
    'weight_decay': sorted({max(0, best_rc['weight_decay'] * 0.1),
                            best_rc['weight_decay'],
                            best_rc['weight_decay'] * 10}),
}

search_train_ds = RealVsAIDataset(search_train, transform=train_transform)
search_val_ds   = RealVsAIDataset(search_val,   transform=eval_transform)

cnn_grid_results = []
keys   = list(cnn_grid.keys())
combos = list(itertools.product(*cnn_grid.values()))
print(f'Running {len(combos)} CNN grid search trials (on {SEARCH_TRAIN} images, 3 epochs each)...\n')

for trial, values in enumerate(combos):
    cfg  = dict(zip(keys, values))
```

```
    tr_l = DataLoader(search_train_ds, batch_size=cfg['batch_size'], shuffle=True,  num_workers=4)
    val_l = DataLoader(search_val_ds,  batch_size=cfg['batch_size'], shuffle=False, num_workers=4)
    torch.manual_seed(SEED)
    model   = CNN(cfg['num_filters'], cfg['dropout'], IMG_SIZE)
    val_acc = quick_train(model, tr_l, val_l,
                          lr=cfg['lr'], weight_decay=cfg['weight_decay'],
                          n_epochs=5, patience=3)
    cfg['val_acc'] = val_acc
    cnn_grid_results.append(cfg)
    print(f'  Trial {trial+1:2d}: val_acc={val_acc:.4f} | '
          f'lr={cfg["lr"]:.2e} wd={cfg["weight_decay"]:.2e}')

cnn_grid_results.sort(key=lambda x: x['val_acc'], reverse=True)
BEST_CNN_CFG = cnn_grid_results[0]
print(f'\n★ Best CNN config: {BEST_CNN_CFG}')
```

```
Best CNN random config: {'num_filters': 16, 'lr': 0.0001, 'weight_decay': 1e-05, 'dropout': 0.2, 'batch_size': 32, 'val_acc': 0.669}

Running 9 CNN grid search trials (on 5000 images, 3 epochs each)...

  Trial  1: val_acc=0.6560 | lr=5.00e-05 wd=1.00e-06
  Trial  2: val_acc=0.6580 | lr=5.00e-05 wd=1.00e-05
  Trial  3: val_acc=0.6520 | lr=5.00e-05 wd=1.00e-04
  Trial  4: val_acc=0.6710 | lr=1.00e-04 wd=1.00e-06
  Trial  5: val_acc=0.6630 | lr=1.00e-04 wd=1.00e-05
  Trial  6: val_acc=0.6680 | lr=1.00e-04 wd=1.00e-04
  Trial  7: val_acc=0.6630 | lr=2.00e-04 wd=1.00e-06
  Trial  8: val_acc=0.6560 | lr=2.00e-04 wd=1.00e-05
  Trial  9: val_acc=0.6610 | lr=2.00e-04 wd=1.00e-04

★ Best CNN config: {'num_filters': 16, 'dropout': 0.2, 'batch_size': 32, 'lr': 0.0001, 'weight_decay': 1.0000000000000002e-06, 'val_a
cc': 0.671}
```

The CNN hyperparameter search was limited by CPU constraints, with each trial running for 5 epochs on a 2,000-image subset. The grid search explored learning rate in the neighbourhood of the best random configuration

---

Section 6 — Training with Best Hyperparameters

Train both models to convergence. Best checkpoint saved automatically to `outputs/`.

In [19]:

```
# 6.0  Training function with best-model saving

def full_train(model, train_loader, val_loader,
               lr, weight_decay, n_epochs=100, patience=8,
               model_name='model'):
    """
    Train to convergence, recording metrics every epoch.
    Saves the best val-loss checkpoint to OUT_DIR/<model_name>_best.pth.
    Restores best weights before returning.
    """
    model     = model.to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimiser = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    # ReduceLROnPlateau halves the learning rate whenever val_loss hasn't improved
    # for 3 consecutive epochs. This helps the model escape flat regions
    # without needing a manually tuned learning rate schedule.
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimiser, mode='min', factor=0.5, patience=3)

    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    no_improve    = 0
    best_weights  = None
    # Save the best checkpoint to disk as a safety net — if the HPC job is
    # killed or times out, the best weights are not lost.
    best_ckpt     = os.path.join(OUT_DIR, f'{HPC_PREFIX}{model_name}_best.pth')

    for epoch in range(1, n_epochs + 1):
        t0 = time.time()
        tr_loss, tr_acc           = train_epoch(model, train_loader, criterion, optimiser)
        vl_loss, vl_acc, _, _     = eval_epoch(model, val_loader, criterion)
        scheduler.step(vl_loss)

        history['train_loss'].append(tr_loss)
        history['val_loss'].append(vl_loss)
        history['train_acc'].append(tr_acc)
        history['val_acc'].append(vl_acc)

        print(f'  Epoch {epoch:3d}/{n_epochs}  '
              f'tr_loss={tr_loss:.4f}  vl_loss={vl_loss:.4f}  '
              f'tr_acc={tr_acc:.4f}  vl_acc={vl_acc:.4f}  '
              f'({time.time()-t0:.1f}s)')

        # Track best performance on val_loss rather than val_accuracy.
        # Loss is a smoother, more informative signal — accuracy can plateau
        # while the model is still improving its calibration.
        if vl_loss < best_val_loss:
            best_val_loss = vl_loss
            no_improve    = 0
            # deepcopy is necessary here — without it, best_weights would just be
            # a reference to the model's current state_dict, which keeps updating
            # as training continues, losing the actual best checkpoint.
            best_weights  = copy.deepcopy(model.state_dict())
            torch.save(best_weights, best_ckpt)

        else:
```

```
            no_improve += 1
            if no_improve >= patience:
                print(f'  Early stopping at epoch {epoch}.')
                break

    # Restore the best checkpoint before returning so the caller always
    # gets the model at peak validation performance, not the final epoch.
    if best_weights is not None:
        model.load_state_dict(best_weights)
        print(f'  Best model restored → {best_ckpt}')
    return history


print('full_train() defined.')
```

```
full_train() defined.
```

In [20]:
```
# 6.1  Train best MLP

print('=' * 60)
print('Training MLP (HOG features)')
print(f'Config: {BEST_MLP_CFG}')
print('=' * 60)

# Train the MLP on the full dataset using the best hyperparameters found
# in Section 5. Max 100 epochs with early stopping patience of 8.
mlp_tr_l, mlp_val_l, mlp_test_l = make_hog_loaders(BEST_MLP_CFG['batch_size'])

best_mlp = MLP(HOG_FEATURE_DIM,
               BEST_MLP_CFG['hidden_dims'],
               BEST_MLP_CFG['dropout']).to(DEVICE)

mlp_history = full_train(
    best_mlp, mlp_tr_l, mlp_val_l,
    lr=BEST_MLP_CFG['lr'], weight_decay=BEST_MLP_CFG['weight_decay'],
    n_epochs=100, patience=8, model_name='mlp'
)
```

```
============================================================
Training MLP (HOG features)
Config: {'hidden_dims': [1024, 512, 256], 'dropout': 0.5, 'batch_size': 128, 'lr': 0.0005, 'weight_decay': 1e-05, 'val_acc': 0.767}
============================================================
  Epoch   1/100  tr_loss=0.4571  vl_loss=0.3698  tr_acc=0.7686  vl_acc=0.8204  (8.1s)
  Epoch   2/100  tr_loss=0.3574  vl_loss=0.3147  tr_acc=0.8256  vl_acc=0.8489  (8.1s)
  Epoch   3/100  tr_loss=0.3117  vl_loss=0.2992  tr_acc=0.8489  vl_acc=0.8534  (8.0s)
  Epoch   4/100  tr_loss=0.2807  vl_loss=0.2791  tr_acc=0.8639  vl_acc=0.8622  (8.1s)
  Epoch   5/100  tr_loss=0.2529  vl_loss=0.2711  tr_acc=0.8777  vl_acc=0.8715  (8.2s)
  Epoch   6/100  tr_loss=0.2316  vl_loss=0.2619  tr_acc=0.8864  vl_acc=0.8756  (8.1s)
  Epoch   7/100  tr_loss=0.2149  vl_loss=0.2566  tr_acc=0.8951  vl_acc=0.8769  (8.2s)
  Epoch   8/100  tr_loss=0.1983  vl_loss=0.2520  tr_acc=0.9021  vl_acc=0.8810  (8.2s)
  Epoch   9/100  tr_loss=0.1858  vl_loss=0.2620  tr_acc=0.9083  vl_acc=0.8802  (8.2s)
  Epoch  10/100  tr_loss=0.1770  vl_loss=0.2565  tr_acc=0.9125  vl_acc=0.8836  (8.0s)
  Epoch  11/100  tr_loss=0.1686  vl_loss=0.2614  tr_acc=0.9158  vl_acc=0.8845  (8.2s)
  Epoch  12/100  tr_loss=0.1619  vl_loss=0.2632  tr_acc=0.9196  vl_acc=0.8840  (8.0s)
  Epoch  13/100  tr_loss=0.1337  vl_loss=0.2664  tr_acc=0.9321  vl_acc=0.8898  (8.2s)
  Epoch  14/100  tr_loss=0.1237  vl_loss=0.2721  tr_acc=0.9371  vl_acc=0.8916  (8.1s)
  Epoch  15/100  tr_loss=0.1172  vl_loss=0.2733  tr_acc=0.9396  vl_acc=0.8924  (8.2s)
  Epoch  16/100  tr_loss=0.1152  vl_loss=0.2741  tr_acc=0.9410  vl_acc=0.8943  (8.3s)
  Early stopping at epoch 16.
  Best model restored → /users/adhp470/neco_project/outputs/HPC_mlp_best.pth
```

In [21]:
```
# 6.2  Train best CNN

print('=' * 60)
print('Training CNN (raw pixels)')
print(f'Config: {BEST_CNN_CFG}')
print('=' * 60)

# Train the CNN on the full dataset using its own best hyperparameters.
# The CNN trains on raw pixels via the image DataLoader (not HOG features).
cnn_tr_l, cnn_val_l, cnn_test_l = make_cnn_loaders(BEST_CNN_CFG['batch_size'])

best_cnn = CNN(BEST_CNN_CFG['num_filters'],
               BEST_CNN_CFG['dropout'],
               IMG_SIZE).to(DEVICE)

cnn_history = full_train(
    best_cnn, cnn_tr_l, cnn_val_l,
    lr=BEST_CNN_CFG['lr'], weight_decay=BEST_CNN_CFG['weight_decay'],
    n_epochs=100, patience=8, model_name='cnn'
)
```

```
============================================================
Training CNN (raw pixels)
Config: {'num_filters': 16, 'dropout': 0.2, 'batch_size': 32, 'lr': 0.0001, 'weight_decay': 1.0000000000000002e-06, 'val_acc': 0.671}
============================================================
  Epoch   1/100  tr_loss=0.5757  vl_loss=0.4797  tr_acc=0.6876  vl_acc=0.7604  (100.1s)
  Epoch   2/100  tr_loss=0.4876  vl_loss=0.4215  tr_acc=0.7512  vl_acc=0.7967  (101.0s)
  Epoch   3/100  tr_loss=0.4429  vl_loss=0.3795  tr_acc=0.7793  vl_acc=0.8144  (99.9s)
  Epoch   4/100  tr_loss=0.4113  vl_loss=0.3757  tr_acc=0.7957  vl_acc=0.8185  (98.0s)
  Epoch   5/100  tr_loss=0.3896  vl_loss=0.3456  tr_acc=0.8092  vl_acc=0.8311  (98.8s)
  Epoch   6/100  tr_loss=0.3714  vl_loss=0.3189  tr_acc=0.8188  vl_acc=0.8458  (101.0s)
  Epoch   7/100  tr_loss=0.3574  vl_loss=0.3268  tr_acc=0.8263  vl_acc=0.8432  (100.2s)
  Epoch   8/100  tr_loss=0.3445  vl_loss=0.3009  tr_acc=0.8327  vl_acc=0.8567  (98.7s)
  Epoch   9/100  tr_loss=0.3341  vl_loss=0.2819  tr_acc=0.8383  vl_acc=0.8628  (98.5s)
  Epoch  10/100  tr_loss=0.3234  vl_loss=0.3070  tr_acc=0.8431  vl_acc=0.8524  (99.2s)
  Epoch  11/100  tr_loss=0.3161  vl_loss=0.2751  tr_acc=0.8479  vl_acc=0.8680  (101.5s)
  Epoch  12/100  tr_loss=0.3061  vl_loss=0.2907  tr_acc=0.8533  vl_acc=0.8615  (98.8s)
  Epoch  13/100  tr_loss=0.2987  vl_loss=0.2603  tr_acc=0.8573  vl_acc=0.8736  (100.1s)
  Epoch  14/100  tr_loss=0.2907  vl_loss=0.2585  tr_acc=0.8613  vl_acc=0.8762  (98.7s)
  Epoch  15/100  tr_loss=0.2845  vl_loss=0.2582  tr_acc=0.8640  vl_acc=0.8772  (100.5s)
```

```
Epoch  16/100  tr_loss=0.2786  vl_loss=0.2528  tr_acc=0.8667  vl_acc=0.8791  (99.9s)
Epoch  17/100  tr_loss=0.2722  vl_loss=0.2489  tr_acc=0.8710  vl_acc=0.8831  (98.7s)
Epoch  18/100  tr_loss=0.2663  vl_loss=0.2350  tr_acc=0.8733  vl_acc=0.8878  (99.3s)
Epoch  19/100  tr_loss=0.2638  vl_loss=0.2343  tr_acc=0.8741  vl_acc=0.8887  (100.0s)
Epoch  20/100  tr_loss=0.2562  vl_loss=0.2274  tr_acc=0.8777  vl_acc=0.8922  (100.8s)
Epoch  21/100  tr_loss=0.2524  vl_loss=0.2323  tr_acc=0.8802  vl_acc=0.8885  (98.8s)
Epoch  22/100  tr_loss=0.2480  vl_loss=0.2315  tr_acc=0.8823  vl_acc=0.8910  (103.0s)
Epoch  23/100  tr_loss=0.2457  vl_loss=0.2175  tr_acc=0.8843  vl_acc=0.8962  (100.4s)
Epoch  24/100  tr_loss=0.2425  vl_loss=0.2165  tr_acc=0.8851  vl_acc=0.8981  (98.7s)
Epoch  25/100  tr_loss=0.2366  vl_loss=0.2210  tr_acc=0.8872  vl_acc=0.8953  (99.2s)
Epoch  26/100  tr_loss=0.2343  vl_loss=0.2200  tr_acc=0.8894  vl_acc=0.8960  (99.3s)
Epoch  27/100  tr_loss=0.2307  vl_loss=0.2321  tr_acc=0.8910  vl_acc=0.8920  (100.5s)
Epoch  28/100  tr_loss=0.2262  vl_loss=0.2105  tr_acc=0.8939  vl_acc=0.8998  (100.1s)
Epoch  29/100  tr_loss=0.2256  vl_loss=0.2200  tr_acc=0.8930  vl_acc=0.8966  (99.6s)
Epoch  30/100  tr_loss=0.2214  vl_loss=0.2073  tr_acc=0.8956  vl_acc=0.9016  (99.7s)
Epoch  31/100  tr_loss=0.2182  vl_loss=0.2036  tr_acc=0.8970  vl_acc=0.9038  (99.5s)
Epoch  32/100  tr_loss=0.2161  vl_loss=0.2074  tr_acc=0.8986  vl_acc=0.9022  (100.5s)
Epoch  33/100  tr_loss=0.2127  vl_loss=0.2073  tr_acc=0.9002  vl_acc=0.9043  (99.6s)
Epoch  34/100  tr_loss=0.2103  vl_loss=0.2062  tr_acc=0.9011  vl_acc=0.9042  (104.8s)
Epoch  35/100  tr_loss=0.2100  vl_loss=0.1978  tr_acc=0.9015  vl_acc=0.9066  (101.2s)
Epoch  36/100  tr_loss=0.2043  vl_loss=0.1954  tr_acc=0.9044  vl_acc=0.9083  (102.0s)
Epoch  37/100  tr_loss=0.2029  vl_loss=0.2200  tr_acc=0.9047  vl_acc=0.8992  (109.4s)
Epoch  38/100  tr_loss=0.2011  vl_loss=0.1950  tr_acc=0.9055  vl_acc=0.9097  (100.4s)
Epoch  39/100  tr_loss=0.2012  vl_loss=0.1928  tr_acc=0.9061  vl_acc=0.9100  (99.6s)
Epoch  40/100  tr_loss=0.1964  vl_loss=0.1986  tr_acc=0.9084  vl_acc=0.9093  (100.5s)
Epoch  41/100  tr_loss=0.1963  vl_loss=0.1902  tr_acc=0.9084  vl_acc=0.9142  (101.2s)
Epoch  42/100  tr_loss=0.1919  vl_loss=0.1887  tr_acc=0.9116  vl_acc=0.9116  (100.7s)
Epoch  43/100  tr_loss=0.1908  vl_loss=0.1906  tr_acc=0.9114  vl_acc=0.9118  (100.1s)
Epoch  44/100  tr_loss=0.1906  vl_loss=0.1870  tr_acc=0.9113  vl_acc=0.9136  (102.2s)
Epoch  45/100  tr_loss=0.1894  vl_loss=0.1848  tr_acc=0.9114  vl_acc=0.9134  (101.5s)
Epoch  46/100  tr_loss=0.1863  vl_loss=0.1845  tr_acc=0.9139  vl_acc=0.9136  (100.8s)
Epoch  47/100  tr_loss=0.1849  vl_loss=0.1856  tr_acc=0.9143  vl_acc=0.9153  (102.5s)
Epoch  48/100  tr_loss=0.1840  vl_loss=0.1971  tr_acc=0.9147  vl_acc=0.9105  (99.9s)
Epoch  49/100  tr_loss=0.1824  vl_loss=0.1863  tr_acc=0.9156  vl_acc=0.9155  (101.4s)
Epoch  50/100  tr_loss=0.1788  vl_loss=0.1942  tr_acc=0.9176  vl_acc=0.9126  (100.3s)
Epoch  51/100  tr_loss=0.1641  vl_loss=0.1775  tr_acc=0.9247  vl_acc=0.9197  (100.2s)
Epoch  52/100  tr_loss=0.1619  vl_loss=0.1769  tr_acc=0.9252  vl_acc=0.9198  (100.9s)
Epoch  53/100  tr_loss=0.1600  vl_loss=0.1736  tr_acc=0.9265  vl_acc=0.9205  (101.9s)
Epoch  54/100  tr_loss=0.1604  vl_loss=0.1721  tr_acc=0.9266  vl_acc=0.9223  (100.2s)
Epoch  55/100  tr_loss=0.1586  vl_loss=0.1744  tr_acc=0.9268  vl_acc=0.9234  (100.1s)
Epoch  56/100  tr_loss=0.1566  vl_loss=0.1736  tr_acc=0.9287  vl_acc=0.9226  (101.6s)
Epoch  57/100  tr_loss=0.1549  vl_loss=0.1723  tr_acc=0.9296  vl_acc=0.9240  (99.7s)
Epoch  58/100  tr_loss=0.1537  vl_loss=0.1711  tr_acc=0.9291  vl_acc=0.9237  (100.1s)
Epoch  59/100  tr_loss=0.1546  vl_loss=0.1694  tr_acc=0.9294  vl_acc=0.9237  (100.6s)
Epoch  60/100  tr_loss=0.1529  vl_loss=0.1706  tr_acc=0.9304  vl_acc=0.9245  (100.4s)
Epoch  61/100  tr_loss=0.1507  vl_loss=0.1680  tr_acc=0.9317  vl_acc=0.9261  (100.7s)
Epoch  62/100  tr_loss=0.1525  vl_loss=0.1705  tr_acc=0.9308  vl_acc=0.9255  (101.4s)
Epoch  63/100  tr_loss=0.1487  vl_loss=0.1757  tr_acc=0.9320  vl_acc=0.9234  (99.6s)
Epoch  64/100  tr_loss=0.1505  vl_loss=0.1742  tr_acc=0.9310  vl_acc=0.9250  (100.1s)
Epoch  65/100  tr_loss=0.1490  vl_loss=0.1767  tr_acc=0.9319  vl_acc=0.9258  (101.1s)
Epoch  66/100  tr_loss=0.1393  vl_loss=0.1671  tr_acc=0.9364  vl_acc=0.9287  (101.0s)
Epoch  67/100  tr_loss=0.1388  vl_loss=0.1668  tr_acc=0.9376  vl_acc=0.9279  (99.2s)
Epoch  68/100  tr_loss=0.1381  vl_loss=0.1688  tr_acc=0.9373  vl_acc=0.9292  (100.5s)
Epoch  69/100  tr_loss=0.1384  vl_loss=0.1668  tr_acc=0.9376  vl_acc=0.9296  (101.3s)
Epoch  70/100  tr_loss=0.1373  vl_loss=0.1635  tr_acc=0.9384  vl_acc=0.9292  (106.5s)
Epoch  71/100  tr_loss=0.1352  vl_loss=0.1682  tr_acc=0.9387  vl_acc=0.9287  (101.9s)
Epoch  72/100  tr_loss=0.1365  vl_loss=0.1658  tr_acc=0.9382  vl_acc=0.9302  (99.8s)
Epoch  73/100  tr_loss=0.1344  vl_loss=0.1649  tr_acc=0.9389  vl_acc=0.9296  (101.7s)
Epoch  74/100  tr_loss=0.1358  vl_loss=0.1642  tr_acc=0.9385  vl_acc=0.9302  (102.7s)
Epoch  75/100  tr_loss=0.1302  vl_loss=0.1638  tr_acc=0.9411  vl_acc=0.9313  (100.6s)
Epoch  76/100  tr_loss=0.1294  vl_loss=0.1639  tr_acc=0.9414  vl_acc=0.9319  (100.7s)
Epoch  77/100  tr_loss=0.1298  vl_loss=0.1643  tr_acc=0.9415  vl_acc=0.9317  (102.1s)
Epoch  78/100  tr_loss=0.1299  vl_loss=0.1648  tr_acc=0.9410  vl_acc=0.9310  (101.3s)
Early stopping at epoch 78.
Best model restored → /users/adhp470/neco_project/outputs/HPC_cnn_best.pth
```

Section 7 — Results and Figures

In [22]:

```
# Figure 2: Training and validation curves

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# plot_both overlays MLP and CNN curves on the same axes — a consistent
# helper used four times (train/val loss and train/val accuracy).
# Putting both models on the same chart makes the comparison immediate.
def plot_both(ax, mlp_vals, cnn_vals, ylabel, title):
    ax.plot(range(1, len(mlp_vals)+1), mlp_vals, label='MLP (HOG)', color='steelblue', lw=2)
    ax.plot(range(1, len(cnn_vals)+1), cnn_vals, label='CNN', color='darkorange', lw=2)
    ax.set_xlabel('Epoch'); ax.set_ylabel(ylabel); ax.set_title(title)
    ax.legend(); ax.grid(True, alpha=0.3)

plot_both(axes[0,0], mlp_history['train_loss'], cnn_history['train_loss'], 'Loss', 'Training Loss')
plot_both(axes[0,1], mlp_history['val_loss'],   cnn_history['val_loss'],   'Loss', 'Validation Loss')
plot_both(axes[1,0], mlp_history['train_acc'],  cnn_history['train_acc'],  'Accuracy', 'Training Accuracy')
plot_both(axes[1,1], mlp_history['val_acc'],    cnn_history['val_acc'],    'Accuracy', 'Validation Accuracy')

fig.suptitle('Figure 2 — Training and Validation Curves: MLP vs CNN',
             fontsize=14, fontweight='bold')
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig2_training_curves.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig2_training_curves.png
```

In [23]:
```
# Test set evaluation

# Run both models on the held-out test set for the final evaluation.
# The test set was never used during training or hyperparameter search,
# so these numbers represent unbiased performance estimates.
criterion = nn.CrossEntropyLoss()

_, mlp_test_acc, mlp_preds, mlp_true = eval_epoch(best_mlp, mlp_test_l, criterion)
_, cnn_test_acc, cnn_preds, cnn_true = eval_epoch(best_cnn, cnn_test_l, criterion)

print(f'\nTest Results')
print('=' * 50)
print(f'MLP test accuracy: {mlp_test_acc:.4f}')
print(f'CNN test accuracy: {cnn_test_acc:.4f}')
print('\nMLP Classification Report:')
print(classification_report(mlp_true, mlp_preds, target_names=['Real', 'AI']))
print('\nCNN Classification Report:')
print(classification_report(cnn_true, cnn_preds, target_names=['Real', 'AI']))
```

```
Test Results
==================================================
MLP test accuracy: 0.8804
CNN test accuracy: 0.9260

MLP Classification Report:
              precision    recall  f1-score   support

        Real       0.92      0.83      0.87     10000
          AI       0.85      0.93      0.89     10000

    accuracy                           0.88     20000
   macro avg       0.88      0.88      0.88     20000
weighted avg       0.88      0.88      0.88     20000


CNN Classification Report:
              precision    recall  f1-score   support

        Real       0.94      0.91      0.92     10000
          AI       0.91      0.94      0.93     10000

    accuracy                           0.93     20000
   macro avg       0.93      0.93      0.93     20000
weighted avg       0.93      0.93      0.93     20000
```

In [24]:
```
# Figure 3: Confusion matrices

# Confusion matrices show where each model makes mistakes — whether it
# tends to misclassify real images as AI, or AI images as real.
# This is more informative than accuracy alone, especially for understanding
# asymmetric error patterns.
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, preds, true, title in [
    (axes[0], mlp_preds, mlp_true, f'MLP (HOG)  acc={mlp_test_acc:.3f}'),
    (axes[1], cnn_preds, cnn_true, f'CNN         acc={cnn_test_acc:.3f}'),
]:
    cm = confusion_matrix(true, preds)
    sns.heatmap(cm, annot=True, fmt='d', ax=ax,
                xticklabels=['Real', 'AI'], yticklabels=['Real', 'AI'], cmap='Blues')
    ax.set_xlabel('Predicted'); ax.set_ylabel('True'); ax.set_title(title)

fig.suptitle('Figure 3 — Test Set Confusion Matrices', fontsize=14, fontweight='bold')
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig3_confusion_matrices.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig3_confusion_matrices.png
```

In [25]:
```
# Figure 4: Side-by-side metric comparison

# Weighted F1 is reported alongside accuracy as a more complete metric —
# it accounts for both precision and recall across classes.
mlp_f1 = f1_score(mlp_true, mlp_preds, average='weighted')
cnn_f1 = f1_score(cnn_true, cnn_preds, average='weighted')

metrics    = ['Accuracy', 'Weighted F1']
mlp_scores = [mlp_test_acc, mlp_f1]
cnn_scores = [cnn_test_acc, cnn_f1]

x, width = np.arange(len(metrics)), 0.35
fig, ax = plt.subplots(figsize=(8, 5))
bars1 = ax.bar(x - width/2, mlp_scores, width, label='MLP (HOG)', color='steelblue')
bars2 = ax.bar(x + width/2, cnn_scores, width, label='CNN',        color='darkorange')
for bar in list(bars1) + list(bars2):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
            f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=10)
ax.set_xticks(x); ax.set_xticklabels(metrics, fontsize=12)
ax.set_ylabel('Score'); ax.set_ylim(0, 1.08); ax.legend(fontsize=11)
ax.set_title('Figure 4 — Test Set Performance Comparison', fontsize=13, fontweight='bold')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig4_comparison.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig4_comparison.png
```

In [26]:
```
# Figure 5: Hyperparameter search scatter plots

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, results, title in [
    (axes[0], mlp_random_results, 'MLP Random Search'),
    (axes[1], cnn_random_results, 'CNN Random Search'),
]:
    # Plot validation accuracy against learning rate across all random trials.
    # This helps visualise whether there's a clear optimal lr range
    # and gives insight into the sensitivity of each model to this parameter.
    lrs  = [r['lr']      for r in results]
    accs = [r['val_acc'] for r in results]
    sc   = ax.scatter(lrs, accs, c=accs, cmap='RdYlGn',
                      s=80, edgecolors='k', lw=0.5, vmin=0.5, vmax=1.0)
    ax.set_xscale('log'); ax.set_xlabel('Learning Rate (log scale)')
    ax.set_ylabel('Validation Accuracy'); ax.set_title(title)
    plt.colorbar(sc, ax=ax, label='Val Acc'); ax.grid(True, alpha=0.3)

fig.suptitle('Figure 5 — Random Search: Val Accuracy vs Learning Rate',
             fontsize=13, fontweight='bold')
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig5_random_search.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig5_random_search.png
```

Section 8 — Model Inspection (Supplementary)

In [27]:
```
# 8.1  CNN first-layer filter visualisation

# Visualise the first conv layer's learned filters as a diagnostic tool.
# These are the low-level detectors the CNN learned — edges, colour blobs,
# and texture patterns. Structured, diverse filters suggest healthy training;
# dead or identical filters would indicate a problem.
filters   = best_cnn.features[0].weight.data.cpu()
n_filters = filters.shape[0]
ncols, nrows = 8, (n_filters + 7) // 8

fig, axes = plt.subplots(nrows, ncols, figsize=(ncols * 1.5, nrows * 1.5))
for i, ax in enumerate(axes.flatten()):
    if i < n_filters:
        f = filters[i]
        # Normalise each filter to [0, 1] for display only — the actual weights
        # are not modified. The 1e-8 epsilon prevents division by zero.
        f = (f - f.min()) / (f.max() - f.min() + 1e-8)
        ax.imshow(f.permute(1, 2, 0).numpy())
    ax.axis('off')

fig.suptitle('CNN First-Layer Learned Filters', fontsize=12)
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig_cnn_filters.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
Saved to /users/adhp470/neco_project/outputs/HPC_fig_cnn_filters.png
```

In [28]:
```
# 8.2  Show CNN misclassified examples

# Visualising misclassified examples helps understand the failure modes —
# whether errors occur on ambiguous images that would fool a human too,
# or on images that should have been easy to classify correctly.
wrong_idx = np.where(cnn_preds != cnn_true)[0]
print(f'CNN misclassified {len(wrong_idx)} / {len(cnn_true)} test images')

n_show = min(10, len(wrong_idx))
class_names = {0: 'Real', 1: 'AI'}

# Visually inspect a handful of images from each class to confirm
# the data loaded correctly and to get a feel for how different
# real and AI-generated images actually look.
fig, axes = plt.subplots(2, 5, figsize=(15, 7))
for ax, idx in zip(axes.flatten(), wrong_idx[:n_show]):
    row = test_sub.iloc[idx]
    img = Image.open(get_full_path(row['filename'])).convert('RGB')
    ax.imshow(img.resize((IMG_SIZE, IMG_SIZE)))
    ax.set_title(
        f'True: {class_names[cnn_true[idx]]}\nPred: {class_names[cnn_preds[idx]]}',
        fontsize=9, color='red'
    )
    ax.axis('off')
for ax in axes.flatten()[n_show:]:
    ax.axis('off')

fig.suptitle('CNN Misclassified Examples', fontsize=12, fontweight='bold')
plt.tight_layout()
out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}fig_misclassified.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved to {out_path}')
```

```
CNN misclassified 1481 / 20000 test images
Saved to /users/adhp470/neco_project/outputs/HPC_fig_misclassified.png
```

Section 9 — Final Summary and Model Saving

In [29]:
```
# Final summary table

# Collect all key metrics and hyperparameters into one summary table.
# Also saved as a CSV so results can be referenced without re-running the notebook.
summary = pd.DataFrame({
    'Model':          ['MLP (HOG)', 'CNN'],
    'Test Accuracy':  [round(mlp_test_acc, 4), round(cnn_test_acc, 4)],
    'Weighted F1':    [round(f1_score(mlp_true, mlp_preds, average='weighted'), 4),
                       round(f1_score(cnn_true, cnn_preds, average='weighted'), 4)],
    'Precision (AI)': [round(precision_score(mlp_true, mlp_preds, pos_label=1), 4),
                       round(precision_score(cnn_true, cnn_preds, pos_label=1), 4)],
    'Recall (AI)':    [round(recall_score(mlp_true, mlp_preds, pos_label=1), 4),
                       round(recall_score(cnn_true, cnn_preds, pos_label=1), 4)],
    'Best LR':        [BEST_MLP_CFG['lr'],          BEST_CNN_CFG['lr']],
    'Best WD':        [BEST_MLP_CFG['weight_decay'], BEST_CNN_CFG['weight_decay']],
    'Dropout':        [BEST_MLP_CFG['dropout'],      BEST_CNN_CFG['dropout']],
})
print('=' * 70)
print('FINAL RESULTS SUMMARY')
print('=' * 70)
print(summary.to_string(index=False))
out_csv = os.path.join(OUT_DIR, f'{HPC_PREFIX}results_summary.csv')
summary.to_csv(out_csv, index=False)
print(f'\nSaved to {out_csv}')
```

```
======================================================================
FINAL RESULTS SUMMARY
======================================================================
    Model  Test Accuracy  Weighted F1  Precision (AI)  Recall (AI)  Best LR  Best WD  Dropout
MLP (HOG)         0.8804       0.8802          0.8473       0.9281   0.0005 0.000010      0.5
      CNN         0.9260       0.9259          0.9130       0.9416   0.0001 0.000001      0.2

Saved to /users/adhp470/neco_project/outputs/HPC_results_summary.csv
```

In [30]:
```
# Save models, scaler, and configs

# Note: the *_best.pth files were already saved during training (prefixed with HPC_).
# This cell saves the final-state copies and auxiliary files.
# Save state_dict (weights only) rather than the full model object.
# This is the recommended PyTorch approach — more portable and
# avoids issues when the class definition changes between saves and loads.
torch.save(best_mlp.state_dict(), os.path.join(OUT_DIR, f'{HPC_PREFIX}best_mlp_final.pth'))
torch.save(best_cnn.state_dict(), os.path.join(OUT_DIR, f'{HPC_PREFIX}best_cnn_final.pth'))

# The scaler must be saved alongside the MLP weights — it captures the
# mean and variance of the training HOG features. Using the wrong scaler
# at inference time would silently shift the input distribution and hurt accuracy.
with open(os.path.join(OUT_DIR, f'{HPC_PREFIX}hog_scaler.pkl'), 'wb') as f:
    pickle.dump(scaler, f)

# Save the best hyperparameter configs as JSON so the test script can
# reconstruct the exact model architectures before loading the weights.
# Without this, there's no reliable way to know what hidden_dims or
# num_filters were used for the saved checkpoint.
with open(os.path.join(OUT_DIR, f'{HPC_PREFIX}best_configs.json'), 'w') as f:
    json.dump({'mlp': BEST_MLP_CFG, 'cnn': BEST_CNN_CFG}, f, indent=2, default=str)

print(f'Saved: {HPC_PREFIX}best_mlp_final.pth, {HPC_PREFIX}best_cnn_final.pth, {HPC_PREFIX}hog_scaler.pkl, {HPC_PREFIX}best_configs.j
print('\nAll output files:')
for fname in sorted(os.listdir(OUT_DIR)):
    if fname.endswith(('.pth', '.pkl', '.json', '.png', '.csv')):
        print(f'  {fname}')
```

```
Saved: HPC_best_mlp_final.pth, HPC_best_cnn_final.pth, HPC_hog_scaler.pkl, HPC_best_configs.json

All output files:
  HPC_best_cnn_final.pth
  HPC_best_configs.json
  HPC_best_mlp_final.pth
  HPC_cnn_best.pth
  HPC_fig1_class_distribution.png
  HPC_fig2_training_curves.png
  HPC_fig3_confusion_matrices.png
  HPC_fig4_comparison.png
  HPC_fig5_random_search.png
  HPC_fig_cnn_filters.png
  HPC_fig_hog_visualisation.png
  HPC_fig_misclassified.png
  HPC_fig_sample_images.png
  HPC_hog_scaler.pkl
  HPC_mlp_best.pth
  HPC_results_summary.csv
  cnn_best.pth
  fig1_class_distribution.png
  fig_hog_visualisation.png
  fig_sample_images.png
  mlp_best.pth
```

Section 10 — Inference Demo

In [31]:
```
# Inference demo: predict a single image

# predict_single_image runs both trained models on one image and plots
# their predicted P(AI-generated) side by side. Useful as a quick sanity
# check that the saved models actually work correctly end-to-end.
```

```
def predict_single_image(img_path):
    img_pil = Image.open(img_path).convert('RGB')

    # MLP path: extract HOG features then scale using the training scaler.
    # reshape(1, -1) converts the 1D feature vector to a 2D array (1 sample)
    # as StandardScaler.transform expects a 2D input.
    hog_feat   = extract_hog_features(img_pil)
    hog_scaled = scaler.transform(hog_feat.reshape(1, -1))
    hog_tensor = torch.tensor(hog_scaled, dtype=torch.float32).to(DEVICE)

    best_mlp.eval()
    with torch.no_grad():
        mlp_logits = best_mlp(hog_tensor)
        # softmax converts raw logits to probabilities; [0] removes the batch dim.
        # mlp_prob[1] is P(AI-generated), mlp_prob[0] is P(real).
        mlp_prob   = torch.softmax(mlp_logits, dim=1)[0]
        mlp_pred   = mlp_logits.argmax(dim=1).item()

    # CNN path: apply the same eval transform used during training, then
    # add a batch dimension with unsqueeze(0) since the model expects [N, C, H, W].
    img_tensor = eval_transform(img_pil).unsqueeze(0).to(DEVICE)
    best_cnn.eval()
    with torch.no_grad():
        cnn_logits = best_cnn(img_tensor)
        cnn_prob   = torch.softmax(cnn_logits, dim=1)[0]
        cnn_pred   = cnn_logits.argmax(dim=1).item()

    label_map = {0: 'REAL', 1: 'AI-GENERATED'}
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
    ax1.imshow(img_pil.resize((IMG_SIZE, IMG_SIZE))); ax1.axis('off'); ax1.set_title('Input Image')
    ax2.bar(['MLP', 'CNN'],
            [mlp_prob[1].item(), cnn_prob[1].item()],
            color=['steelblue', 'darkorange'])
    ax2.set_ylabel('P(AI-generated)'); ax2.set_ylim(0, 1)
    ax2.axhline(0.5, color='red', linestyle='--', label='Decision boundary')
    ax2.legend(); ax2.set_title('Predicted Probability of Being AI-Generated')
    plt.tight_layout()
    out_path = os.path.join(OUT_DIR, f'{HPC_PREFIX}demo_prediction.png')
    plt.savefig(out_path, dpi=150, bbox_inches='tight')
    plt.show()
    print(f'Saved → {out_path}')
    print(f'MLP: {label_map[mlp_pred]}  (P(AI) = {mlp_prob[1]:.2%})')
    print(f'CNN: {label_map[cnn_pred]}  (P(AI) = {cnn_prob[1]:.2%})')


# Run the demo on the first test image as a quick end-to-end check.
# Demo using the first test image
demo_path = get_full_path(test_sub.iloc[0]['filename'])
predict_single_image(demo_path)
```

```
Saved → /users/adhp470/neco_project/outputs/HPC_demo_prediction.png
MLP: AI-GENERATED  (P(AI) = 99.85%)
CNN: AI-GENERATED  (P(AI) = 96.05%)
```